

Hashing

Hashing is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access.

Hash functions are a fundamental concept in computer science and play a crucial role in various applications such as data storage, retrieval, and cryptography.

Types of Hash Functions

There are many hash functions that use numeric or alphanumeric keys. This article focuses on discussing different hash functions:

1. Division Method.
2. Multiplication Method
3. Mid-Square Method
4. Folding Method
5. Cryptographic Hash Functions
6. Universal Hashing
7. Perfect Hashing

Division Method

The division method involves dividing the key by a prime number and using the remainder as the hash value.

$$h(k) = k \bmod m$$

Where k is the key and m is a prime number

Mid-Square Method

In the mid-square method, the key is squared, and the middle digits of the result are taken as the hash value.

Steps:

1. Square the key.
2. Extract the middle digits of the squared value.

Folding Method

The folding method involves dividing the key into equal parts, summing the parts, and then taking the modulo with respect to m .

Steps:

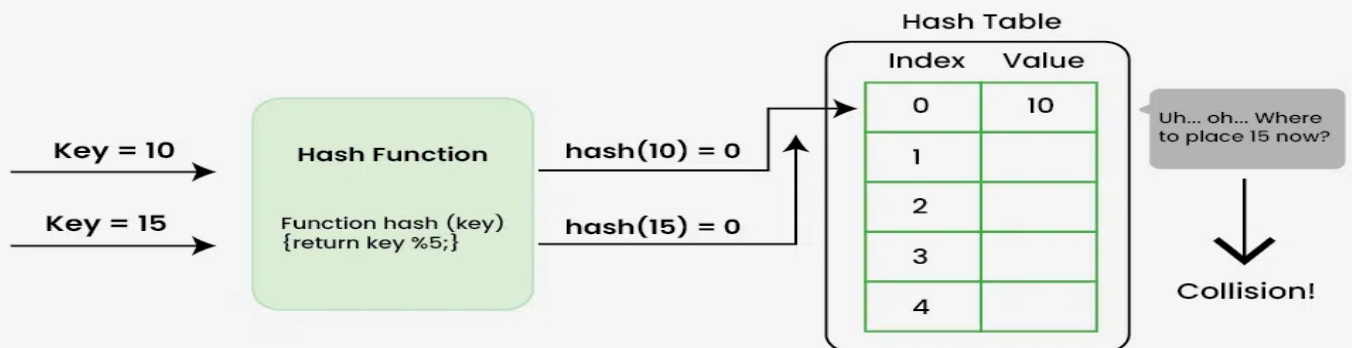
1. Divide the key into parts.
2. Sum the parts.
3. Take the modulo m of the sum.

Collision Resolution Techniques

In Hashing, hash functions were used to generate hash values. The hash value is used to create an index for the keys in the hash table. The hash function may return the same hash value for two or more keys. When two or more keys have the same hash value, a collision happens. To handle this collision, we use Collision Resolution Techniques.

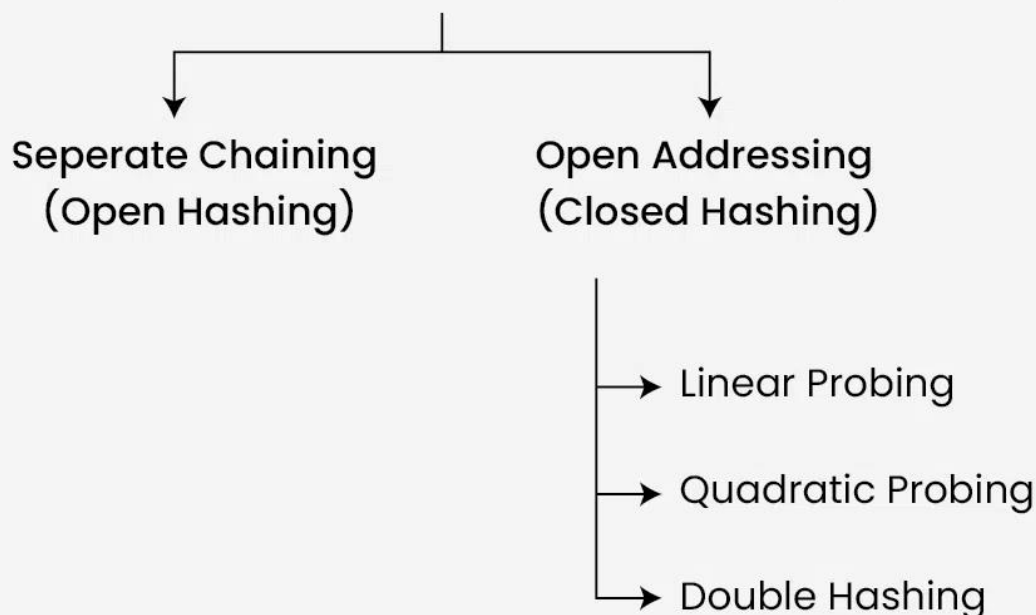


Collision in Hashing





Collision Resolution Techniques



Linear Probing in Hash Tables

Let **hash(x)** be the slot index computed using a hash function and **S** be the table size .

Fucntion: $H(\text{key}) = \text{key} \% \text{table_size}$,

The function used for rehashing is as follows: $\text{rehash}(\text{key}) = (n+1) \% \text{table-size}$.

If slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(\text{key}) + 1) \% S$

If $(\text{hash}(x) + 1) \% S$ is also full, then we try $(\text{hash}(\text{key}) + 2) \% S$

If $(\text{hash}(x) + 2) \% S$ is also full, then we try $(\text{hash}(\text{key}) + 3) \% S$

Given key: 10, 3, 6, 16, 17, 19

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
key													

Insert the key 10, Calculate the hash index: $h(10) = 10 \% 13 = 10$.

the slot index 10 is empty so we insert 10 there

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
key											10		

Insert the key 3, Calculate the hash index: $h(3) = 3 \% 13 = 3$.

the slot index 3 is empty so we insert 3 there

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
key				3							10		

Insert the key 6, Calculate the hash index: $h(6) = 6 \% 13 = 6$.

the slot index 6 is empty so we insert 6 there

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
key				3			6				10		

Insert the key 16, Calculate the hash index: $h(16)=16\%13=3$.

the slot index 3 is not empty. Linear probing means we move to the next slot until we find the empty slot. We continue from index 3 and we find an empty slot at index 4. So we insert 16 there

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
key				3	16		6				10		

Insert the key 17, Calculate the hash index: $h(17)=17\%13=4$.

the slot index 4 is not empty. Linear probing means we move to the next slot until we find the empty slot. We continue from index 4 and we find an empty slot at index 5. So we insert 17 there

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
key				3	16	17	6				10		

Insert the key 19, Calculate the hash index: $h(19)=19\%13=6$.

the slot index 6 is not empty. Linear probing means we move to the next slot until we find the empty slot. We continue from index 6 and we find an empty slot at index 7. So we insert 19 there

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
key				3	16	17	6	19			10		